

```

        icode == IRET : valM;
        # Default: Use incremented PC
        1 : valP;
    ];

```

6. SEQ 小结

现在我们已经浏览了 Y86-64 处理器的一个完整的设计。可以看到，通过将执行每条不同指令所需的步骤组织成一个统一的流程，就可以用很少量的各种硬件单元以及一个时钟来控制计算的顺序，从而实现整个处理器。不过这样一来，控制逻辑就必须要在这些单元之间路由信号，并根据指令类型和分支条件产生适当的控制信号。

SEQ 唯一的问题就是它太慢了。时钟必须非常慢，以使信号能在一个周期内传播所有的阶段。让我们来看看处理一条 `ret` 指令的例子。在时钟周期起始时，从更新过的 PC 开始，要从指令内存中读出指令，从寄存器文件中读出栈指针，ALU 将栈指针加 8，为了得到程序计数器的下一个值，还要从内存中读出返回地址。所有这一切都必须在这个周期结束之前完成。

这种实现方法不能充分利用硬件单元，因为每个单元只在整个时钟周期的一部分时间内才被使用。我们会看到引入流水线能获得更好的性能。

4.4 流水线的通用原理

在试图设计一个流水线化的 Y86-64 处理器之前，让我们先来看看流水线化的系统的一些通用属性和原理。对于曾经在自助餐厅的服务线上工作过或者开车通过自动汽车清洗线的人，都会非常熟悉这种系统。在流水线化的系统中，待执行的任务被划分成了若干个独立的阶段。在自助餐厅，这些阶段包括提供沙拉、主菜、甜点以及饮料。在汽车清洗中，这些阶段包括喷水和打肥皂、擦洗、上蜡和烘干。通常都会允许多个顾客同时经过系统，而不是要等到一个用户完成了所有从头至尾的过程才让下一个开始。在一个典型的自助餐厅流水线上，顾客按照相同的顺序经过各个阶段，即使他们并不需要某些菜。在汽车清洗的情况中，当前面一辆汽车从喷水阶段进入擦洗阶段时，下一辆就可以进入喷水阶段了。通常，汽车必须以相同的速度通过这个系统，避免撞车。

流水线化的一个重要特性就是提高了系统的吞吐量(throughput)，也就是单位时间内服务的顾客总数，不过它也会轻微地增加延迟(latency)，也就是服务一个用户所需要的时间。例如，自助餐厅里的一个只需要甜点的顾客，能很快通过一个非流水线化的系统，只在甜点阶段停留。但是在流水线化的系统中，这个顾客如果试图直接去甜点阶段就有可能招致其他顾客的愤怒了。

4.4.1 计算流水线

让我们把注意力放到计算流水线上来，这里的“顾客”就是指令，每个阶段完成指令执行的一部分。图 4-32a 给出了一个很简单的非流水线化的硬件系统例子。它是由一些执行计算的逻辑以及一个保存计算结果的寄存器组成的。时钟信号控制在每个特定的时间间隔加载寄存器。CD 播放器中的译码器就是这样的一个系统。输入信号是从 CD 表面读出的位，逻辑电路对这些位进行译码，产生音频信号。图中的计算块是用组合逻辑来实现的，意味着信号会穿过一系列逻辑门，在一定时间的延迟之后，输出就成为了输入的某个函数。

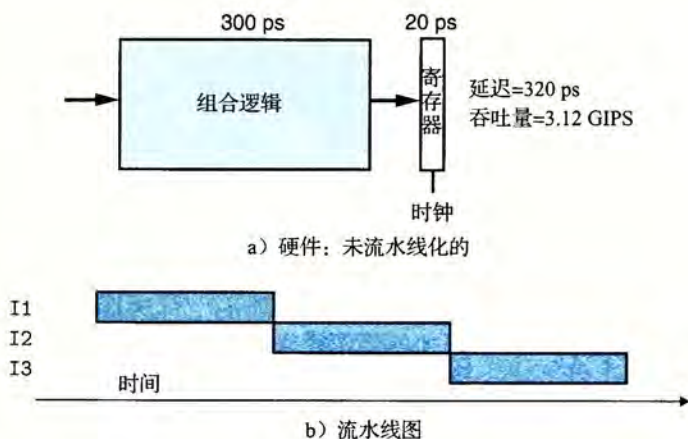


图 4-32 非流水线化的计算硬件。每个 320ps 的周期内，系统用 300ps 计算组合逻辑函数，20ps 将结果存到输出寄存器中

在现代逻辑设计中，电路延迟以微微秒或皮秒(picosecond，简写成“ps”)，也就是 10^{-12} 秒为单位来计算。在这个例子中，我们假设组合逻辑需要 300ps，而加载寄存器需要 20ps。图 4-32 还给出了一种时序图，称为流水线图(pipeline diagram)。在图中，时间从左向右流动。从上到下写着一组操作(在此称为 I1、I2 和 I3)。实心的长方形表示这些指令执行的时间。这个实现中，在开始下一条指令之前必须完成前一个。因此，这些方框在垂直方向上并没有相互重叠。下面这个公式给出了运行这个系统的最大吞吐量：

$$\text{吞吐量} = \frac{1 \text{ 条指令}}{(20 + 300) \text{ ps}} \cdot \frac{1000 \text{ ps}}{1 \text{ ns}} \approx 3.12 \text{ GIPS}$$

我们以每秒千兆条指令(GIPS)，也就是每秒十亿条指令，为单位来描述吞吐量。从头到尾执行一条指令所需要的时间称为延迟(latency)。在此系统中，延迟为 320ps，也就是吞吐量的倒数。

假设将系统执行的计算分成三个阶段(A、B 和 C)，每个阶段需要 100ps，如图 4-33 所

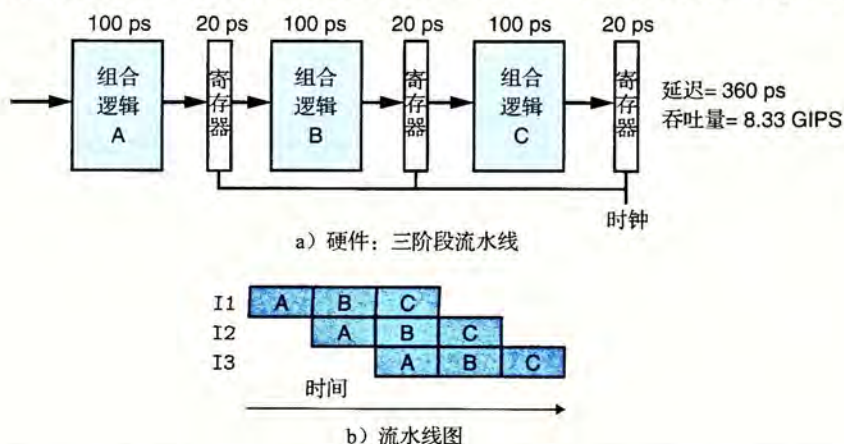


图 4-33 三阶段流水线化的计算硬件。计算被划分为三个阶段 A、B 和 C。每经过一个 120ps 的周期，每条指令就行进通过一个阶段

⊖ $1 \text{ ns} = 10^{-9} \text{ s}$ 。

示。然后在各个阶段之间放上流水线寄存器(pipeline register), 这样每条指令都会按照三步经过这个系统, 从头到尾需要三个完整的时钟周期。如图 4-33 中的流水线图所示, 只要 I1 从 A 进入 B, 就可以让 I2 进入阶段 A 了, 依此类推。在稳定状态下, 三个阶段都应该是活动的, 每个时钟周期, 一条指令离开系统, 一条新的进入。从流水线图中第三个时钟周期就能看出这一点, 此时, I1 是在阶段 C, I2 在阶段 B, 而 I3 是在阶段 A。在这个系统中, 我们将时钟周期设为 $100+20=120\text{ps}$, 得到的吞吐量大约为 8.33 GIPS。因为处理一条指令需要 3 个时钟周期, 所以这条流水线的延迟就是 $3 \times 120 = 360\text{ps}$ 。我们将系统吞吐量提高到原来的 $8.33/3.12 = 2.67$ 倍, 代价是增加了一些硬件, 以及延迟的少量增加 ($360/320 = 1.12$)。延迟变大是由于增加的流水线寄存器的时间开销。

4.4.2 流水线操作的详细说明

为了更好地理解流水线是怎样工作的, 让我们来详细看看流水线计算的时序和操作。图 4-34 给出了前面我们看到过的三阶段流水线(图 4-33)的流水线图。就像流水线图上方指明的那样, 流水线阶段之间的指令转移是由时钟信号来控制的。每隔 120ps , 信号从 0 上升至 1, 开始下一组流水线阶段的计算。

图 4-35 跟踪了时刻 $240 \sim 360$ 之间的电路活动, 指令 I1 经过阶段 C, I2 经过阶段 B, 而 I3 经过阶段 A。就在时刻 240(点 1)时钟上升之前, 阶段 A 中计算的指令 I2 的值已经到达第一个流水线寄存器的输入, 但是该寄存器的状态和输出还保持为指令 I1 在阶段 A 中计算的值。指令 I1 在阶段 B 中计算的值已经到达第

二个流水线寄存器的输入。当时钟上升时, 这些输入被加载到流水线寄存器中, 成为寄存器的输出(点 2)。另外, 阶段 A 的输入被设置成发起指令 I3 的计算。然后信号传播通过各个阶段的组合逻辑(点 3)。就像图中点 3 处的曲线化的波阵面(curved wavefront)表明的那样, 信号可能以不同的速率通过各个不同的部分。在时刻 360 之前, 结果值到达流水线寄存器的输入(点 4)。当时刻 360 时钟上升时, 各条指令会前进经过一个流水线阶段。

从这个对流水线操作详细的描述中, 我们可以看到减缓时钟不会影响流水线的行为。信号传播到流水线寄存器的输入, 但是直到时钟上升时才会改变寄存器的状态。另一方面, 如果时钟运行得太快, 就会有灾难性的后果。值可能会来不及通过组合逻辑, 因此当时钟上升时, 寄存器的输入还不是合法的值。

根据对 SEQ 处理器时序的讨论(4.3.3 节), 我们看到这种在组合逻辑块之间采用时钟寄存器的简单机制, 足够控制流水线中的指令流。随着时钟周而复始地上升和下降, 不同的指令就会通过流水线的各个阶段, 不会相互干扰。

4.4.3 流水线的局限性

图 4-33 的例子给出了一个理想的流水线化的系统, 在这个系统中, 我们可以将计算分成三个相互独立的阶段, 每个阶段需要的时间是原来逻辑需要时间的三分之一。不幸的是, 会出现其他一些因素, 降低流水线的效率。

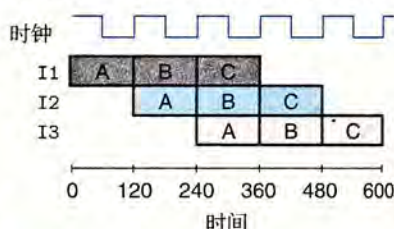


图 4-34 三阶段流水线的时序。时钟信号的上升沿控制指令从一个流水线阶段移动到下一个阶段

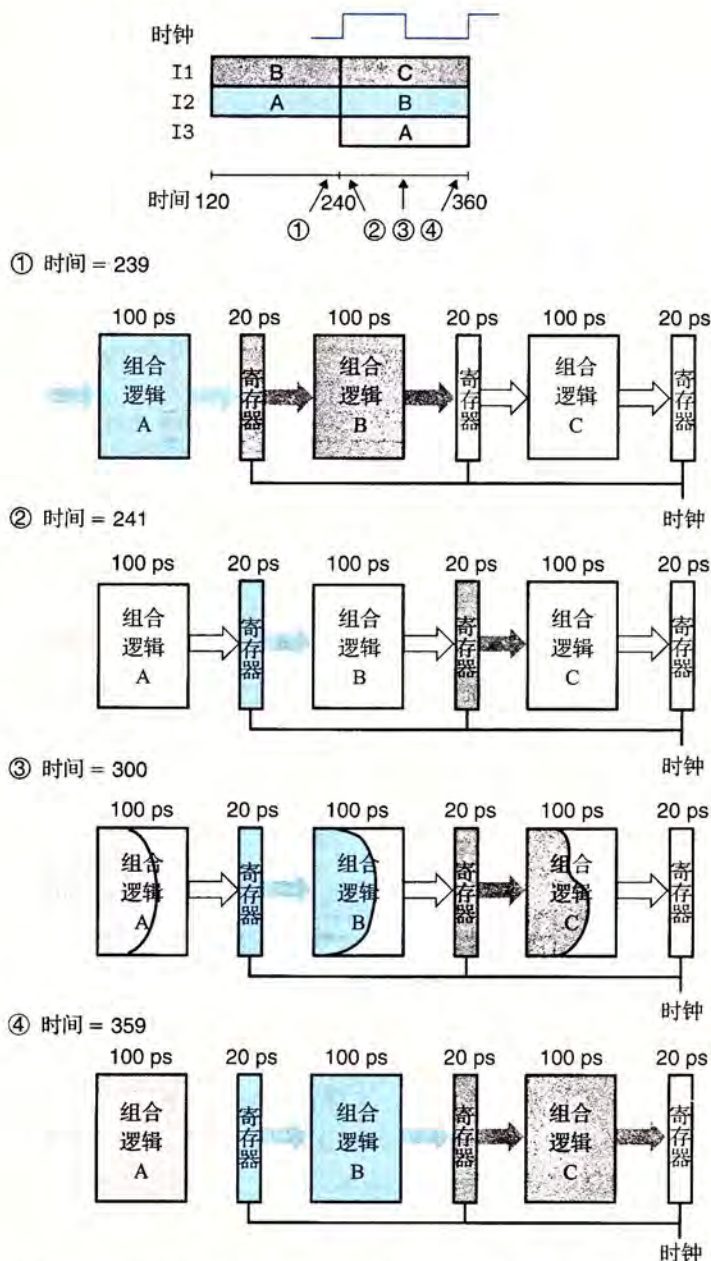
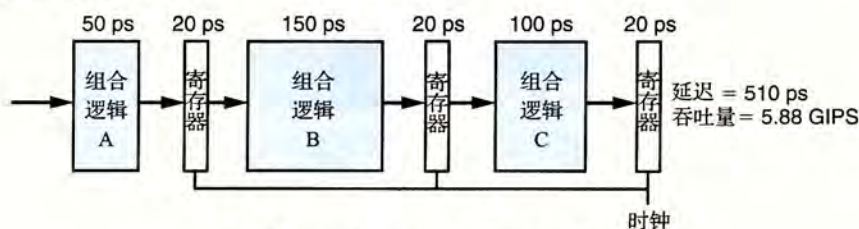


图 4-35 流水线操作的一个时钟周期。在时刻 240(点 1)时钟上升之前, 指令 I1 和 I2 已经完成了阶段 B 和 A。在时钟上升后, 这些指令开始传送到阶段 C 和 B, 而指令 I3 开始经过阶段 A(点 2 和 3)。就在时钟开始再次上升之前, 这些指令的结果就会传到流水线寄存器的输入(点 4)

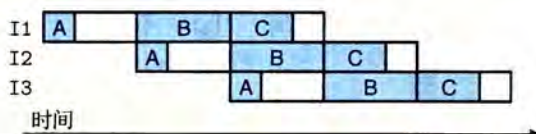
1. 不一致的划分

图 4-36 展示的系统中和前面一样, 我们将计算划分为了三个阶段, 但是通过这些阶段的延迟从 50ps 到 150ps 不等。通过所有阶段的延迟和仍然为 300ps。不过, 运行时钟的速率是由最慢的阶段的延迟限制的。流水线图表明, 每个时钟周期, 阶段 A 都会空闲(用白色方框表示)100ps, 而阶段 C 会空闲 50ps。只有阶段 B 会一直处于活动状态。我们必须将时钟周期设为 $150 + 20 = 170\text{ps}$, 得到吞吐量为 5.88 GIPS。另外, 由于时钟周期减慢

了, 延迟也增加到了 510ps。



a) 硬件: 三阶段流水线, 不一致的阶段延迟

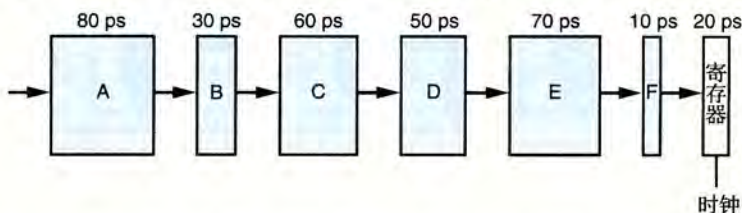


b) 流水线图

图 4-36 由不一致的阶段延迟造成的流水线技术的局限性。系统的吞吐量受最慢阶段的速度所限制

对硬件设计者来说, 将系统计算设计计划分成一组具有相同延迟的阶段是一个严峻的挑战。通常, 处理器中的某些硬件单元, 如 ALU 和内存, 是不能被划分成多个延迟较小的单元的。这就使得创建一组平衡的阶段非常困难。在设计流水线化的 Y86-64 处理器中, 我们不会过于关注这一层次的细节, 但是理解时序优化在实际系统设计中的重要性还是非常重要的。

练习题 4.28 假设我们分析图 4-32 中的组合逻辑, 认为它可以分成 6 个块, 依次命名为 A~F, 延迟分别为 80、30、60、50、70 和 10ps, 如下图所示:



在这些块之间插入流水线寄存器, 就得到这一设计的流水线化的版本。根据在哪里插入流水线寄存器, 会出现不同的流水线深度(有多少个阶段)和最大吞吐量的组合。假设每个流水线寄存器的延迟为 20ps。

- 只插入一个寄存器, 得到一个两阶段的流水线。要使吞吐量最大化, 该在哪里插入寄存器呢? 吞吐量和延迟是多少?
- 要使一个三阶段的流水线的吞吐量最大化, 该将两个寄存器插在哪里呢? 吞吐量和延迟是多少?
- 要使一个四阶段的流水线的吞吐量最大化, 该将三个寄存器插在哪里呢? 吞吐量和延迟是多少?
- 要得到一个吞吐量最大的设计, 至少要有几个阶段? 描述这个设计及其吞吐量和延迟。

2. 流水线过深, 收益反而下降

图 4-37 说明了流水线技术的另一个局限性。在这个例子中, 我们把计算分成了 6 个阶段, 每个阶段需要 50ps。在每对阶段之间插入流水线寄存器就得到了一个六阶段流水线。这个系统的最小时钟周期为 $50 + 20 = 70\text{ps}$, 吞吐量为 14.29 GIPS。因此, 通过将流

水线的阶段数加倍,我们将性能提高了 $14.29/8.33=1.71$ 。虽然我们将每个计算时钟的时间缩短了两倍,但是由于通过流水线寄存器的延迟,吞吐量并没有加倍。这个延迟成了流水线吞吐量的一个制约因素。在我们的新设计中,这个延迟占到了整个时钟周期的28.6%。

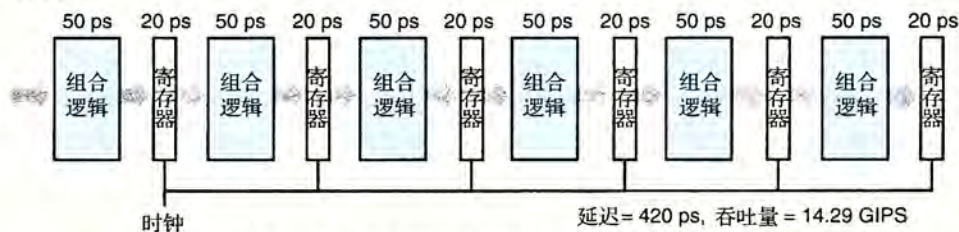


图 4-37 由开销造成的流水线技术的局限性。在组合逻辑被分成较小的块时,由寄存器更新引起的延迟就成为了一个限制因素

为了提高时钟频率,现代处理器采用了很深的(15 或更多的阶段)流水线。处理器架构师将指令的执行划分成很多非常简单的步骤,这样一来每个阶段的延迟就很小。电路设计者小心地设计流水线寄存器,使其延迟尽可能得小。芯片设计者也必须小心地设计时钟传播网络,以保证时钟在整个芯片上同时改变。所有这些都是设计高速微处理器面临的挑战。

练习题 4.29 让我们来看看图 4-32 中的系统,假设将它划分成任意数量的流水线阶段 k ,每个阶段有相同的延迟 $300/k$,每个流水线寄存器的延迟为 20ps。

- 系统的延迟和吞吐量写成 k 的函数是什么?
- 吞吐量的上限等于多少?

4.4.4 带反馈的流水线系统

到目前为止,我们只考虑一种系统,其中传过流水线的对象,无论是汽车、人或者指令,相互都是完全独立的。但是,对于像 x86-64 或 Y86-64 这样执行机器程序的系统来说,相邻指令之间很可能是相关的。例如,考虑下面这个 Y86-64 指令序列:

```

1  irmovq $50, %rax
2  addq %rax, %rbx
3  mrmovq 100(%rbx), %rdx
  
```

Diagram illustrating data dependencies between instructions. Arrows show that the result of the first instruction (irmovq) is used by the second (addq), and the result of the second is used by the third (mrmovq).

在这个包含三条指令的序列中,每对相邻的指令之间都有数据相关(data dependency),用带圈的寄存器名字和它们之间的箭头来表示。irmovq 指令(第 1 行)将它的结果存放在 %rax 中,然后 addq 指令(第 2 行)要读这个值;而 addq 指令将它结果存放在 %rbx 中, mrmovq 指令(第 3 行)要读这个值。

另一种相关是由于指令控制流造成的顺序相关。来看看下面这个 Y86-64 指令序列:

```

1  loop:
2      subq %rdx,%rbx
3      jne targ
4      irmovq $10,%rdx
5      jmp loop
6  targ:
7      halt
  
```


jne 指令(第 3 行)产生了一个控制相关(control dependency), 因为条件测试的结果会决定要执行的新指令是 irmovq 指令(第 4 行)还是 halt 指令(第 7 行)。在我们的 SEQ 设计中, 这些相关都是由反馈路径来解决的, 如图 4-22 的右边所示。这些反馈将更新了寄存器值向下传送到寄存器文件, 将新的 PC 值向下传送到 PC 寄存器。

图 4-38 举例说明了将流水线引入含有反馈路径的系统中的危险。在原来的系统(图 4-38a)中, 每条指令的结果都反馈给下一条指令。流水线图(图 4-38b)就说明了这个情况, I1 的结果成为 I2 的输入, 依此类推。如果试图以最直接的方式将它转换成一个三阶段流水线(图 4-38c), 我们将改变系统的行为。如图 4-38c 所示, I1 的结果成为 I4 的输入。为了通过流水线技术加速系统, 我们改变了系统的行为。

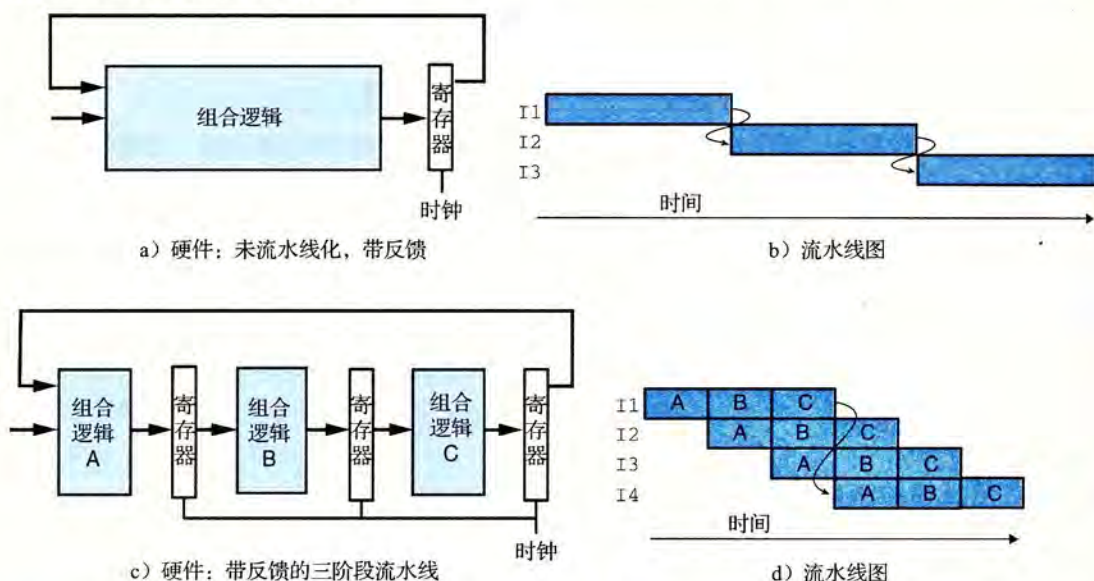


图 4-38 由逻辑相关造成的流水线技术的局限性。在从未流水线化的带反馈的系统 a 转化到流水线化的系统 c 的过程中, 我们改变了它的计算行为, 可以从两个流水线图(b 和 d)中看出来

当我们将流水线技术引入 Y86-64 处理器时, 必须正确处理反馈的影响。很明显, 像图 4-38 中的例子那样改变系统的行为是不可接受的。我们必须以某种方式来处理指令间的数据和控制相关, 以使得到的行为与 ISA 定义的模型相符。

4.5 Y86-64 的流水线实现

我们终于准备好要开始本章的主要任务——设计一个流水线化的 Y86-64 处理器。首先, 对顺序的 SEQ 处理器做一点小的改动, 将 PC 的计算挪到取指阶段。然后, 在各个阶段之间加上流水线寄存器。到这个时候, 我们的尝试还不能正确处理各种数据和控制相关。不过, 做一些修改, 就能实现我们的目标——一个高效的、流水线化的实现 Y86-64 ISA 的处理器。

4.5.1 SEQ+: 重新安排计算阶段

作为实现流水线化设计的一个过渡步骤, 我们必须稍微调整一下 SEQ 中五个阶段的顺序, 使得更新 PC 阶段在一个时钟周期开始时执行, 而不是结束时才执行。只需要对整体硬件结构做最小的改动, 对于流水线阶段中的活动的时序, 它能工作得更好。我们称这